TP Integrador · Clasificador de gastos con IA

Trabajo Práctico Integrador · Seminario de Actualización · Tec. Sup. en Cs. de Datos e IA · IFTS N.º 18

Autora: Araceli Amherdt

A. El problema

1. ¿Qué problema querés resolver y para quién? ¿Por qué un modelo de IA y no unas reglas escritas a mano?

Mi app clasifica gastos escritos en lenguaje natural en seis categorías: comida, transporte, servicios, salud, ocio y educación. La persona escribe el gasto como lo anotaría en el celular ("cargué la SUBE", "pagué la luz") y la app le devuelve la categoría más probable y las dos que le siguen.

Está pensada para cualquier persona que quiera llevar registro de sus gastos sin completar formularios ni elegir la categoría a mano cada vez.

Un `if` con palabras clave no alcanza porque el mismo gasto se escribe de infinitas maneras, incluyendo oraciones muy cortas, lunfardos o incluso ironía.

Para cubrir esto con reglas tendría que prever cada palabra, sigla y forma de decir, y la lista nunca terminaría. Un modelo de lenguaje, en cambio, interpreta el significado de la frase completa. En la app publicada, por ejemplo, "birras con los pibes" se clasifica como ocio con 87 %, sin que yo haya escrito ninguna regla sobre "birras".

B. El modelo

2. ¿Qué modelos consideraste y por qué te quedaste con uno?

Me quedé con `MoritzLaurer/mDeBERTa-v3-base-mnli-xnli` porque acertó 13 de 15 frases (87 %), contra 6 de 15 (40 %) del otro modelo, y además pesa cuatro veces menos.

Busqué en el Hub modelos con la tarea *Zero-Shot Classification* que entendieran español. Elegí zero-shot porque me deja definir mis propias categorías sin entrenar nada, comparé los dos candidatos con las mismas 15 frases, y las mismas categorías.

Información de la Model Card.

Repositorio: MoritzLaurer/mDeBERTa-v3-base-mnli-xnli

Tarea: Zero-shot classification (basada en NLI)

Idiomas: Multilingüe: preentrenado en 100 idiomas

Datos de entrenamiento: El modelo fue preentrenado por Microsoft en el conjunto de datos multilingüe CC100, luego se ajustó con el conjunto de datos de desarrollo XNLI y el conjunto de datos de entrenamiento MNLI. El conjunto de desarrollo XNLI consta de 2490 textos traducidos profesionalmente del inglés a otros 14 idiomas.

Pesos: 558MB

De la *model card* de mDeBERTa tomé también su precisión en XNLI: 0,845 en español contra 0,883 en inglés. La de xlm-roberta recomienda traducir la plantilla de hipótesis si se trabaja en un solo idioma; por eso usé "Este gasto es de {}." en lugar de la plantilla en inglés.

El resultado me sorprendió: esperaba que el modelo más grande fuera mejor, y fue peor. Para mi problema, más parámetros no significó más aciertos. La muestra es chica (15 frases), pero la diferencia es lo bastante amplia para decidir. Además, como la app corre en un plan gratuito que vuelve a cargar el modelo cada vez que se despierta, un modelo más liviano mejora directamente la espera de quien la usa.

3. ¿Dónde se equivoca?

mDeBERTa falló en [frase 1] y [frase 2], y acertó con poca seguridad en otras frases con humor o jerga.

Frase	Esperada	Respondió	Puntaje	Segunda opción
hice las compras en el supermercado	Comida	Servicios	0.591	Transporte
Netflix	Ocio	Servicios	0.914	Ocio
otra vez al dentista, mi billetera llora	salud	salud (acierto dudoso)	0.581	Transporte (0.34)

Creo que la causa principal está en los datos con los que fue entrenado. XNLI se armó con traducciones profesionales de textos en inglés: oraciones completas y bien escritas, sin lunfardo argentino ("lucas", "bondi", "el chino") ni humor. Por eso, cuando una frase usa jerga o ironía, el modelo se apoya en palabras sueltas. En "otra vez al dentista, mi billetera llora", "mi billetera llora" no le aporta nada y "otra vez al" suena a ir a algún lado, lo que podría explicar el 34 % de transporte.

4. Tu app, ¿está entrenando o infiriendo?

Mi app infiere: usa un modelo que ya fue entrenado por otra persona y no cambia sus pesos en ningún momento.

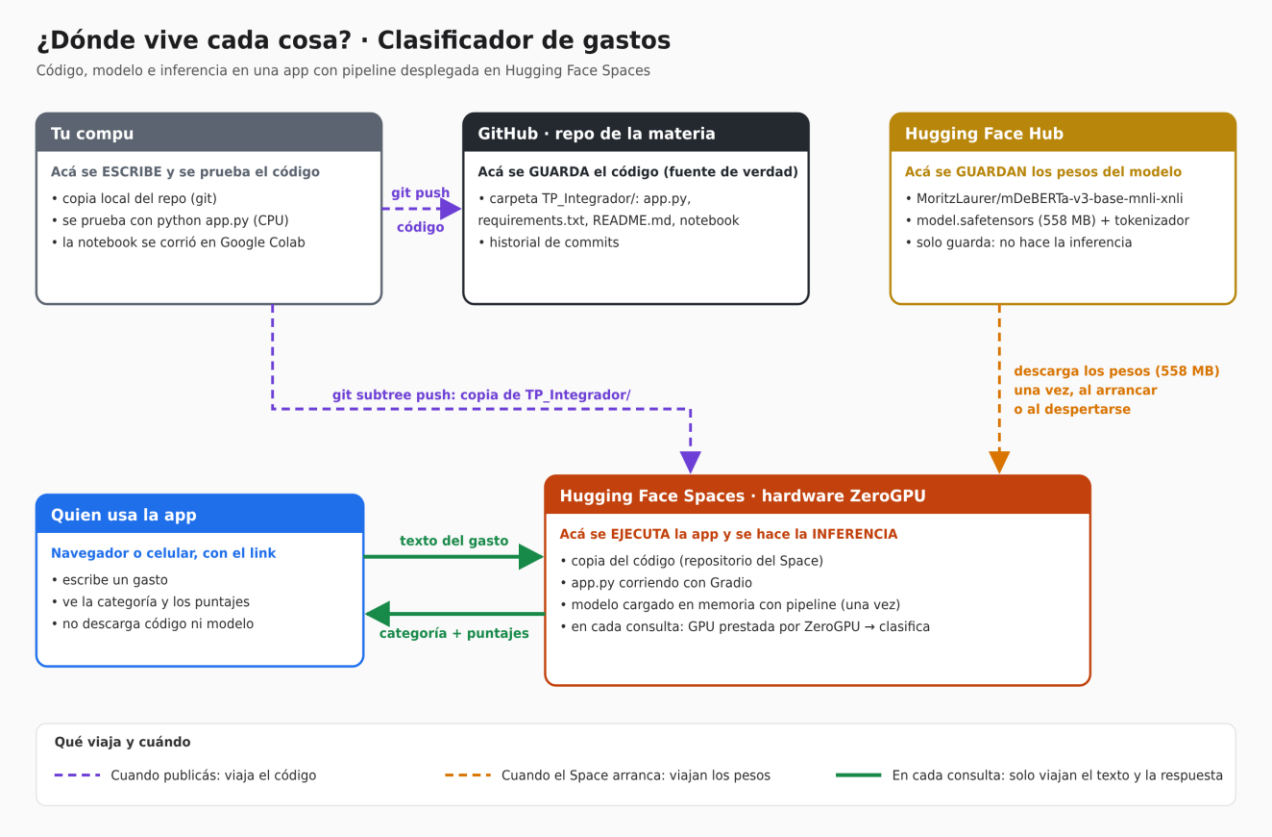
Entrenar es ajustar los pesos de un modelo mostrándole muchos ejemplos con la respuesta correcta. Eso ya lo hizo el autor de mDeBERTa con millones de pares de oraciones de XNLI y MNLI. Inferir es usar esos pesos fijos para responder sobre un dato nuevo.

En mi caso, cuando alguien escribe "cargué la SUBE", el modelo arma una hipótesis por categoría ("Este gasto es de transporte.") y calcula con sus pesos fijos qué tan probable es cada una. Nunca le mostré un solo gasto de ejemplo así como tampoco aprende de las consultas: la frase número mil se clasifica con los mismos pesos que la primera.

C. Dónde corre cada cosa

5. ¿Dónde vive el código y dónde vive el modelo?

El código se guarda en GitHub y se ejecuta en Hugging Face Spaces; los pesos se guardan en el Hugging Face Hub y la inferencia se hace en el Space.



Aunque el Hub y el Space son de Hugging Face, cumplen roles distintos: el Hub solo guarda los pesos y el Space los usa. Tampoco viaja todo siempre. El código viaja solo cuando publico un cambio, y los pesos, solo cuando el Space arranca o se despierta. En cada consulta viajan únicamente el texto del gasto y la respuesta.

6. ¿Usaste pipeline o InferenceClient en la app publicada?

Usé `pipeline`, porque con `InferenceClient` la app no habría funcionado el día de la corrección.

	`pipeline` (elegido)	`InferenceClient`
Dónde corre el modelo	En el Space	En los servidores de Hugging Face
Qué necesita	Memoria para el modelo (558 MB)	Internet, token y créditos
Costo para mi cuenta gratuita	Ninguno por consulta	Menos de USD 0,10 de créditos por mes, ya agotados

	`pipeline` (elegido)	`InferenceClient`
Carga del modelo	20,2 s, una vez por arranque	No hay: el modelo ya está cargado en el servidor
Cada inferencia	Promedio de 15 s en Colab	No se pudo medir: error 402

En la notebook medí que con `pipeline` la carga tarda 20,2 s y cada inferencia un promedio de 15 s: la carga es lo caro, pero ocurre una sola vez. Por eso en `app.py` cargo el modelo fuera de la función decorada con `@spaces.GPU` y dentro dejo solo la clasificación. El costo de esta elección es que, cuando el Space se duerme por inactividad, la primera persona que entra tiene que esperar mientras se vuelve a cargar el modelo.

Si hubiera elegido `InferenceClient`, la app sería más liviana: no necesitaría instalar `transformers` ni `torch` ni cargar 558 MB. Pero cada consulta gastaría créditos de mi cuenta. Las cuentas gratuitas reciben menos de USD 0,10 por mes, los míos ya se agotaron (error `402 Payment Required`) y se renuevan en noviembre, después de la entrega. La app estaría caída en la corrección, sin importar que el código estuviera bien.

D. El deploy

7. ¿Dónde deployaste y por qué ahí?

Deployé en Hugging Face Spaces, un PaaS pensado para apps de Gradio con modelos, porque es la única opción gratuita que encontré donde un modelo de 558 MB y `torch` entran sin administrar un servidor.

Con un PaaS subo el código y la plataforma se encarga del resto: instala lo que pide `requirements.txt`, arranca la app y le da un link público. Hubo un cambio importante: desde julio de 2026, las cuentas gratuitas no pueden crear Spaces de Gradio con CPU Basic, así que el mío usa ZeroGPU. Eso me obligó a marcar la función de clasificación con `@spaces.GPU`: la GPU se presta solo mientras corre esa función y se descuenta de la cuota diaria de cada visitante.

Tipo de hosting	Por qué lo descarté
Hosting compartido	Está pensado para páginas web y PHP, con poca memoria y procesos cortos. No permite cargar `torch` ni dejar un modelo de 558 MB en memoria.
VPS	Me da un servidor completo, pero tengo que instalar y mantener todo yo (sistema, Python, seguridad, reinicios). Uno con memoria suficiente para el modelo es pago.
Serverless	Cada función arranca al recibir una consulta y se apaga después. Tendría que cargar el modelo (20 s en Colab) en cada arranque en frío, y suele tener límites de memoria y tamaño bajos para un modelo así.

Mi app necesita un servidor que quede encendido porque el modelo tiene que estar cargado en memoria para responder rápido. Cargarlo tarda 20 s y clasificar una frase, 15 s: si el servidor se apagara

entre consulta y consulta, cada persona esperaría la carga completa. El Space se duerme después de un tiempo sin uso, y por eso la primera consulta después de despertarse es la única lenta.

E. Lo que se rompió

8. Errores reales

Documento tres errores, cada uno en un lugar distinto: dos en Hugging Face y uno en la plataforma de deploy.

Error 1: 401 Unauthorized al usar InferenceClient

```
HfHubHTTPError: Client error '401 Unauthorized' for url
'https://router.huggingface.co/hf-inference/models/MoritzLaurer/mDeBERTa-v3-base-
mnli-xnli'
Invalid username or password.
```

- **Dónde apareció:** en Hugging Face. El pedido salió de Colab y llegó al servidor (se ve en la URL), que lo rechazó.
- **Qué lo causaba:**El token guardado no era valido..
- **Cómo lo resolví:**
    a. En Hugging Face generé un token nuevo en *Settings → Access Tokens*
    b. En Colab, abrí *Secrets* (ícono de llave) y guardé el token con el nombre `HF_TOKEN`, para no escribirlo en el código.
    c. Lo leí con `from google.colab import userdata` y `userdata.get("HF_TOKEN")`.

Error 2: 402 Payment Required

```
HfHubHTTPError: Client error '402 Payment Required' for url
'https://router.huggingface.co/hf-inference/models/MoritzLaurer/mDeBERTa-v3-base-
mnli-xnli'
You have no remaining credits. Purchase pre-paid credits to continue using
Inference Providers. Alternatively, subscribe to PRO to get monthly included credits.
```

- **Dónde apareció:** en Hugging Face (Inference Providers), justo después de resolver el 401. Que el error cambiara confirmó que el token ya funcionaba.
- **Qué lo causaba:** se agotaron los créditos mensuales gratuitos de la cuenta (menos de USD 0,10). Se renuevan en noviembre, y un token nuevo no los renueva porque son de la cuenta.
- **Cómo lo resolví:** no se puede resolver sin pagar, así que cambié el diseño.
    a. Revisé en *Billing* cuándo se renuevan los créditos y la fecha era en noviembre.
    b. Decidí usar `pipeline` en la app, para que no dependa de créditos.
    c. En la notebook dejé la celda de `InferenceClient` capturando el error.

Error 3: Configuration error en el Space

```
Configuration error
Missing configuration in README
```

- **Dónde apareció:** en la plataforma de deploy (Hugging Face Spaces), después del primer `git subtree push`.
- **Qué lo causaba:** el mensaje apuntaba al README del Space, pero la causa estaba en mi compu: el `README.md` se había creado pero nunca se había guardado en el editor antes del commit, así que llegó vacío.

- **Cómo lo resolví:**
    a. En el Space, en *Files* → *README.md*, vi que el archivo estaba vacío.
    b. Confirmé qué se había commiteado con `git show HEAD:TP_Integrador/README.md`: tampoco tenía nada.
    c. Pegué el bloque de configuración entre `---` en la primera línea del README (`sdk: gradio`, `sdk_version`, `app_file: app.py`) y guardé el archivo.
    d. Verifiqué con `git diff` que el cambio existía.
    e. Hice commit, `git push` y, aparte, `git subtree push --prefix TP_Integrador space main`, porque `git push` solo sube a GitHub.

9. Límites de mi solución

La app sirve para mostrar la idea, pero hoy no aguantaría un uso masivo y depende de condiciones que no controlo.

- **Si la usan 1000 personas a la vez:** el Space es un solo servidor con el modelo cargado una vez, y ZeroGPU presta GPU por turnos. Las consultas harían fila y la espera crecería con cada persona. Además, cada visitante gasta su propia cuota diaria de GPU.
- **Si Hugging Face cambia el plan gratuito:** ya pasó durante este trabajo. CPU Basic dejó de ser gratuito para Spaces nuevos y tuve que adaptar la app a ZeroGPU. Un nuevo cambio podría dejarla caída sin que yo toque el código.
- **Límites del modelo:** siempre elige una categoría, aunque lo escrito no sea un gasto, porque reparte el 100 % entre las seis. También falla o duda con lunfardo, humor y frases muy cortas.

Para que sea un producto real, cambiaría tres cosas:

1. **Hardware pago o un servidor propio**, para solucionar las limitaciones.
2. **Un umbral de confianza,** si la mejor categoría queda por debajo de, por ejemplo, 50 %, pedirle a la persona que la elija. Esas respuestas servirían para evaluar mejor el modelo.
3. **Guardar los gastos y sumar montos y fechas**, que es lo que de verdad necesita quien lleva sus cuentas.

F. Vocabulario

10. Cinco líneas de app.py

```
DISPOSITIVO = 0 if torch.cuda.is_available() else -1
clasificador = pipeline("zero-shot-classification", model=MODELO, device=DISPOSITIVO)
resultado = clasificador(texto, candidate_labels=CATEGORIAS,
hypothesis_template=PLANTILLA)
demo = gr.Interface(fn=clasificar, inputs=gr.Textbox(label="Escribí un gasto"),
outputs=gr.Label(label="Categoría", num_top_classes=3))
demo.launch()
```

Clase: gr.Interface (línea 4). Una clase es un molde que define cómo es un tipo de objeto y qué puede hacer. gr.Interface es el molde de Gradio para armar una interfaz: dice qué entra, qué sale y qué función se ejecuta en el medio. En la misma línea hay otras dos clases: gr.Textbox (la caja de texto) y gr.Label (el resultado con barras).

Instancia: demo (línea 4). Una instancia es un objeto concreto creado a partir de una clase. Cuando escribo demo = gr.Interface(...), uso el molde para crear mi interfaz en particular, con mi función y mis componentes, y la guardo en la variable demo. Si la clase es el plano de una casa, demo es la casa construida.

Método: launch() (línea 5). Un método es una función que pertenece a un objeto y se llama con un punto. En demo.launch(), le pido a mi instancia demo que se ponga en marcha: arranca el servidor web y la app queda disponible en el navegador.

Atributo: cuda en torch.cuda (línea 1). Un atributo es un valor guardado dentro de otro objeto, al que también se accede con un punto. La diferencia con un método es que no se ejecuta: se consulta. cuda es un atributo del módulo torch que agrupa todo lo relacionado con la GPU. Sobre él llamo a is_available(), que devuelve True si hay una GPU disponible.

Parámetro y argumento: model=MODELO (línea 2). El parámetro es el nombre que la función espera recibir; el argumento es el valor concreto que le paso. En model=MODELO, model es el parámetro de pipeline y MODELO es el argumento, que vale "MoritzLaurer/mDeBERTa-v3-base-mnli-xnli". En la línea 3 pasa lo mismo con candidate_labels=CATEGORIAS: candidate_labels es el parámetro y la lista CATEGORIAS es el argumento.

Valor de retorno: resultado (línea 3). El valor de retorno es lo que una función devuelve cuando termina. Al llamar a clasificador(...) con un gasto, me devuelve un diccionario con las claves sequence, labels y scores, y lo guardo en la variable resultado.

Enlaces

- **App:** huggingface.co/spaces/AraceliAmherdt/Clasificador_gastos
- **Carpeta del TP en el repo:** https://github.com/AraceliAmherdt/Proyecto_seminario/tree/main/TP_Integrador

Uso de IA generativa

Usé Claude (Anthropic) como asistente para las siguientes tareas:

- Entender cómo funciona la clasificación zero-shot y buscar posibles modelos candidatos.
- Diagnosticar los errores 401, 402 y de configuración del Space, y conectar GitHub con el Space usando `git subtree`.
- Proponer el código de `app.py`, y la estructura de la notebook.
- Dibujar el esquema de la pregunta 5.
- Redactar un primer borrador de este informe a partir de mis resultados, que después revisé y completé.
- Realizar una primer PPT que después también revise y complete.

Las decisiones, la ejecución del código, el deploy, las pruebas y la interpretación de los resultados son mías.